

DDL Tracker

面向 Q 群通知过载的日程提醒自动化

GuotaoHe

2026 年 6 月

目录

1. 问题与目标
2. 实现过程
3. 接口与成果

目录

1. 问题与目标

2. 实现过程

3. 接口与成果

待解决问题：Q 群通知过多

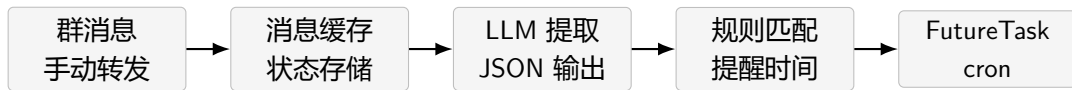
现实痛点

- Q 群里课程、考试、报名、作业通知混在一起，DDL 容易被刷走。
- DDL 表达不统一：绝对时间、相对时间、口语化提醒规则同时存在。
- 手动从聊天切到日历创建提醒，步骤多、打断强、容易漏。

目标是把“看到一条群消息”变成“可追踪、可提醒、可恢复的日程任务”。

- 通过用户手动转发消息给 Agent，降低创建日程的交互成本。
- 通过 AstrBot FutureTask 创建一次性 cron 提醒。
- 通过插件状态记录避免重复创建和重启丢失。

设计逻辑：从消息到提醒



可识别

从自然语言里提取标题、类型、截止时间、原始文本。

可去重

用标题和时间生成指纹，避免相同 DDL 反复创建任务。

可回写

FutureTask 创建后把任务名写回 DDL 状态，支持查询和恢复。

目录

1. 问题与目标

2. 实现过程

3. 接口与成果

技术栈与开发框架

运行栈

- Python 3.10+
- AstrBot Plugin API
- OneBot / aiocqhttp
- FutureTask

AI 与数据

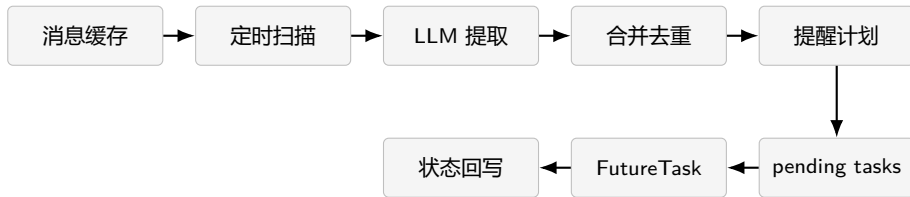
- LLM Provider
- Function Calling
- JSON 状态文件
- 指纹去重

开发工具

- opencode
- Codex
- PowerShell
- XeLaTeX / Beamer

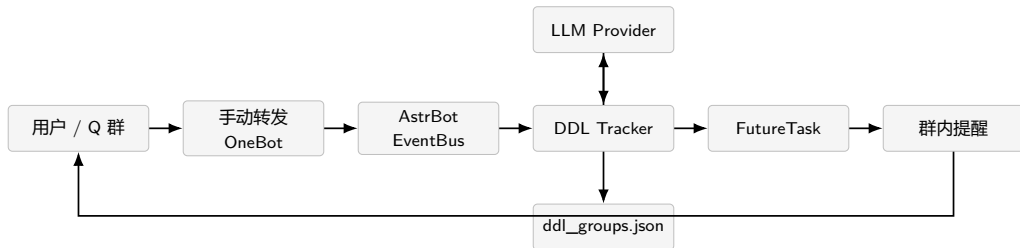
插件入口只负责组合 Mixin 与注册命令；业务逻辑按职责拆分，便于持续扩展和维护。

主要运行思路：后台扫描与任务同步



- 当前主 Agent 工具链已能自动同步 FutureTask。
- 后台扫描负责生成 pending 状态，提醒任务由 FutureTask 统一调度。
- 失败时保留 pending 状态，下次扫描继续重试，避免任务丢失。

组件图：插件内外协作



插件负责识别、去重、计划与状态；LLM 负责语义结构化；FutureTask 负责可靠调度。

目录

1. 问题与目标

2. 实现过程

3. 接口与成果

用户接口与 AI 接口

用户接口

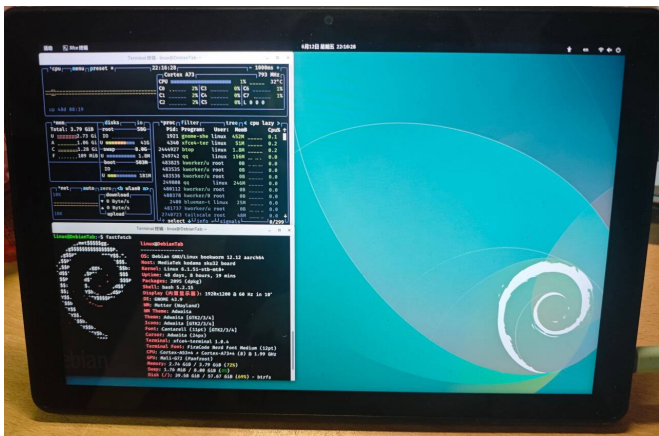
- /ddl_on: 开启群 DDL 跟踪。
- /ddl_extract: 手动整理最近消息。
- /ddl_status: 查看 DDL、规则、任务状态。
- /ddl_nearest: 查看最近截止事项。

AI Function Calling

- ddl_extract_recent_messages
- ddl_set_type_reminder
- ddl_get_pending_future_tasks
- ddl_mark_future_task_created
- future_task

pending 任务携带 future_task_arguments; 创建成功后用 mark_created_arguments 回写绑定状态。

运行设备展示：刷成 Debian 的平板



部署环境

- Debian GNU/Linux
- AArch64 平板设备
- AstrBot 常驻运行
- QQ / OneBot 接入群消息

低功耗设备常驻后台，适合承担群消息监听与提醒调度。

成果展示



项目地址与阶段成果

项目地址

`github.com/GHe0000`
`/astrbot_plugin_ddl_tracker`

已完成

- 群消息记录、DDL 提取、去重与状态持久化。
- 分类提醒规则与 FutureTask 待创建计划。
- 主 Agent 工具链自动同步 future_task。

下一步

- 增加失败重试和状态页。
- 优化分类提醒规则。
- 完善部署文档与演示脚本。

让“群消息提醒”变成自动化日程，而不是手动整理工作。